基于地震数据的Spark数据处理与分析

https://dblab.xmu.edu.cn/blog/2738/

本案例针对全球重大地震数据进行分析，采用Python为编程语言，采用Hadoop存储数据，采用Spark对数据进行处理分析，并对结果进行数据可视化。

一、 实验环境搭建

（1）Linux：Ubuntu 16.04
（2）Hadoop：3.1.3（安装教程）
（3）Spark：2.4.0（安装教程）
（4）Python：Anaconda3（Anaconda及Jupyter Notebook安装教程）
（5）运行环境：Jupyter Notebook

此外，还需要安装plotly用于绘制地图，安装wordcloud用于绘制词云图。

```
conda install plotly
pip install -i https://
pypi.tuna.tsinghua.edu.cn/simple wordcloud
```

Shell 命令

本博客提供了本实验的全部源代码，请到百度网盘下载，下载以后用Jupyter Notebook打开代码文件，调试运行。
下载百度网盘地址为：https://pan.baidu.com/s/1_Trxk1Dbs6R716fx2Vh3jA 提取码: 2hza

二、 数据预处理

数据来自和鲸社区的1965-2016全球重大地震数据，文件名为`earthquake.csv`包括23412条地震数据，其中有很多属性大部分缺失且本次实验用不到，将其手动删除，只保留Date, Time, Latitude, Longitude, Type, Depth, Magnitude这七个属性。
数据集下载百度网盘地址为：https://pan.baidu.com/s/1_Trxk1Dbs6R716fx2Vh3jA 提取码: 2hza

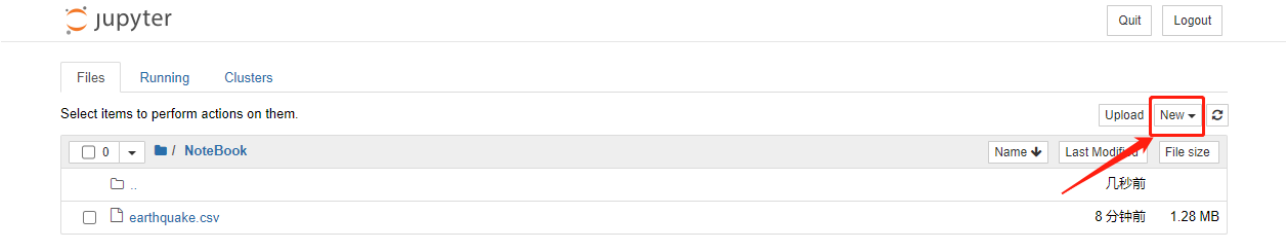
本博客中的Python代码均运行在Jupyter Notebook上。

在用户目录下创建一个`NoteBook`文件夹，将`earthquake.csv`文件放入`NoteBook`中，接着打开Jupyter Notebook。

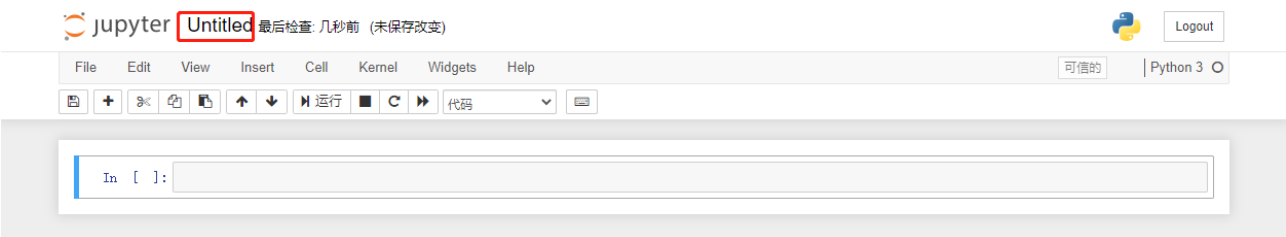
```
cd NoteBook
jupyter notebook
```

Shell 命令

此时会自动打开浏览器并进入Jupyter Notebook的主页，点击右上角的new按钮创建一个新的Python3文件。



这样就进入了Jupyter Notebook执行环境，可以点击左上角红框处给这个文件重命名。





在代码框中输入代码，运行后可以直接显示结果。首先从文件中读取数据：

```python
import pandas as pd
import numpy as np

rawData = pd.read_csv('~/NoteBook/earthquake.csv')
rawData.head(10)
```

	Date	Time	Latitude	Longitude	Type	Depth	Magnitude
0	01/02/1965	13:44:18	19.246	145.616	Earthquake	131.6	6.0
1	01/04/1965	11:29:49	1.863	127.352	Earthquake	80.0	5.8
2	01/05/1965	18:05:58	-20.579	-173.972	Earthquake	20.0	6.2
3	01/08/1965	18:49:43	-59.076	-23.557	Earthquake	15.0	5.8
4	01/09/1965	13:32:50	11.938	126.427	Earthquake	15.0	5.8
5	01/10/1965	13:36:32	-13.405	166.629	Earthquake	35.0	6.7
6	01/12/1965	13:32:25	27.357	87.867	Earthquake	20.0	5.9
7	01/15/1965	23:17:42	-13.309	166.212	Earthquake	35.0	6.0
8	01/16/1965	11:32:37	-56.452	-27.043	Earthquake	95.0	6.0
9	01/17/1965	10:43:17	-24.563	178.487	Earthquake	565.0	5.8

```python
rawData.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23412 entries, 0 to 23411
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Date       23412 non-null  object 
 1   Time       23412 non-null  object 
 2   Latitude   23412 non-null  float64
 3   Longitude  23412 non-null  float64
 4   Type       23412 non-null  object 
 5   Depth      23412 non-null  float64
 6   Magnitude  23412 non-null  float64
dtypes: float64(4), object(3)
memory usage: 1.3+ MB
```

可见这些属性中都没有缺失值，但是可能还存在一些格式不一致的现象，需要对其进行格式转换。

1. 清洗Date和Time数据

Pandas提供了to_datetime()方法，可以将不同的日期格式转换为datetime 类型。设置errors='coerce'参数，遇到不能转换的格式时将其置为NaN。

```python
rawData['Structed Date'] = pd.to_datetime(rawData['Date'],

format = "%m/%d/%Y",

errors='coerce').dt.date
rawData['Structed Date'].loc[rawData['Structed Date'].isnull()]
```

```
3378     NaT
7512     NaT
20650    NaT
Name: Structed Date, dtype: object
```

找到无法正常转换的日期，手动对它们进行修改。

```
rawData['Date'].loc[[3378,7512,20650]]
```

Python

```
3378     1975-02-23T02:58:41.000Z
7512     1985-04-28T02:53:41.530Z
20650    2011-03-13T02:23:34.520Z
Name: Date, dtype: object
```

```
rawData.loc[3378, 'Structed Date'] = '1975-02-23'
rawData.loc[7512, 'Structed Date'] = '1985-04-28'
rawData.loc[20650, 'Structed Date'] = '2011-03-13'
del rawData['Date']
```

Python

接着按照同样地步骤对Time数据进行清洗。

```
# 对时间进行格式转换
rawData['Structed Time'] = pd.to_datetime(rawData['Time'],
format = "%H:%M:%S",
errors='coerce').dt.time
rawData['Structed Time'].loc[rawData['Structed Time'].isnull()]
rawData['Time'].loc[[3378,7512,20650]]
rawData.loc[3378, 'Structed Time'] = '2:58:41'
rawData.loc[7512, 'Structed Time'] = '2:53:41'
rawData.loc[20650, 'Structed Time'] = '2:23:34'
del rawData['Time']
```

Python

修改完成后，将列名重新改回Date和Time。

```
rawData.rename(columns={'Structed Date':
'Date', 'Structed Time': 'Time'},
               inplace=True)
```

Python

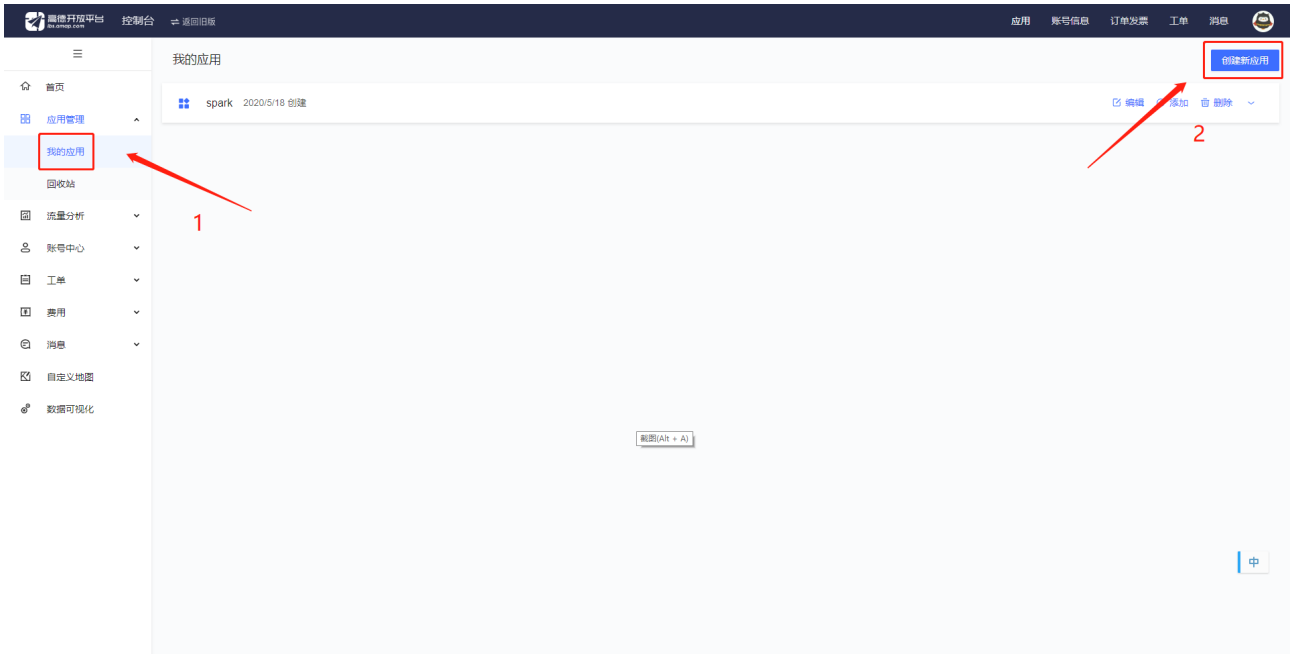
2. 根据经纬度获取地名

高德地图的逆地理编码API服务可以根据经纬度查询出该位置的地名信息。首先需要在高德开放平台注册成为个人开发者；



登录之后，进入右上角的控制台-->应用管理-->我的应用，在里面创建新应用。


高德开放平台
阿里巴巴集团旗下
为什么选择高德
产品介绍
解决方案
开发支持
服务升级
控制台
概述
入门指南
开发指南
获取Key
地理/逆地理编码
路径规划
行政区域查询
搜索POI
IP定位
静态地图
坐标转换
天气查询
输入提示
交通态势
地理围栏
轨迹纠偏
天气对照表
流量限制说明
错误码说明
常见问题
更新日志
相关下载
您现在的位置：开发 > Web服务 API > 概述
概述
最后更新时间: 2020年01月03日
Web服务API简介
高德Web服务API向开发者提供HTTP接口，开发者可通过这些接口使用各类型的地理数据服务，返回结果支持JSON和XML格式。
Web服务API对所有用户开放。使用本组服务之前，需要申请应用Key。不同类型用户可获取不同的数据访问能力。
搜索
路径规划
行驶距离测量
功能介绍及体验
通过下面的demo，可以直观的体验我们提供的服务，点击下拉框，可切换需要体验的服务。
IP定位
参数
值
备注
必选
ip
114.247.50.2
需要搜索的IP地址（仅支持国内）
否


高德开放平台
控制台
返回旧版
应用
账号信息
订单发票
工单
消息
我的应用
创建新应用
首页
应用管理
我的应用
回收站
流量分析
账号中心
工单
费用
消息
自定义地图
数据可视化
spark
2020/5/18 创建
编辑
添加
删除
1
2
截图(Alt + A)


新建应用
* 应用名称：
spark
* 应用类型：
其他
取消
新建

点击新建，新建成功后如图点击添加；



这样就得到一个Key用来给经纬度转地名，具体如何在编程中使用可以参考地理/逆地理编码API文档。

经过测试，这个API对于中国境外的坐标不能准确地返回结果，因此只获取在中国境内的坐标信息。坐标在中国境外时，就返回一个空值。如果坐标在海上，就获取海域的名称（如南海）；如果在陆地上，就获取省份的名称。

编写程序为每个坐标获取地名信息，并将其存到Area列中。这个过程大概需要花费10分钟。

```
import requests
import json

def getProvince(lon, lat):
    u1 = 'http://restapi.amap.com/v3/geocode/
regeo?output=json&'
    key = '&key=**********'    // 星号处填入刚才
在高德开放平台获取到的Key
    location = 'location=' + str(lon) + ','+
str(lat)
    url = u1 + location + key
    res = requests.get(url)
    json_data = json.loads(res.text)
    regeoinfo = json_data['regeocode']
['addressComponent']

    if 'country' in regeoinfo and
regeoinfo['country'] == '中国':
        if 'province' in regeoinfo and
regeoinfo['province']:
            return regeoinfo['province']
        elif 'seaArea' in regeoinfo and
regeoinfo['seaArea']:
            return regeoinfo['seaArea']

    return None


for i in range(23412):
```

```
    lon = rawData.loc[i, 'Longitude']
    lat = rawData.loc[i, 'Latitude']
    rawData.loc[i, 'Area'] = getProvince(lon, lat)
```
Python

查看获取到的地名信息。

```
rawData["Area"].unique()
```
Python

```
array([nan, '南海', None, '东海', '新疆维吾尔自治区', '西藏自治区', '台湾省', '云南省', '河北省',
       '四川省', '青海省', '渤海', '广东省', '江苏省', '辽宁省', '内蒙古自治区', '天津市', '黑龙江省',
       '山东省', '甘肃省', '黄海', '山西省', '吉林省', '贵州省'], dtype=object)
```

3. 清洗Type数据

查看Type数据中是否存在异常值。

```
rawData.Type.unique()
```
Python

```
array(['Earthquake', 'Nuclear Explosion', 'Explosion', 'Rock Burst'],
      dtype=object)
```

可见中一共有四种不同的类型，且不存在异常值。

至此，数据清洗工作完成，将数据保存到`earthquake_cleaned.csv`文件中。使用`utf-8`编码防止Spark读取时出现中文乱码，设置`index=False`防止将第一列索引保存至文件中。

```
rawData.to_csv("earthquake_cleaned.csv",
               encoding='utf-8',
               index=False)
```
Python

上传到HDFS以便Spark进行数据分析。

```
# 首先启动hadoop
cd /usr/local/hadoop
./sbin/start-dfs.sh
# 将文件上传到HDFS
```

```shell
./bin/hdfs dfs -mkdir -p /user/[username]  # username处填入当前Linux系统登录的用户名
./bin/hdfs dfs -mkdir input
./bin/hdfs dfs -put ~/NoteBook/earthquake_cleaned.csv  input
```

Shell 命令

三、 Spark数据分析

用Spark读取earthquake_cleaned.csv文件创建DataFrame。

```python
from pyspark import SparkContext,SparkConf
from pyspark.sql import SparkSession
spark = SparkSession.builder.config(conf = SparkConf()).getOrCreate()

df = spark.read.csv("input/earthquake_cleaned.csv",
                    header=True,
                    inferSchema=True)
df = df.repartition(1)
df.show(10)
```

Python

Latitude	Longitude	Type	Depth	Magnitude	Date	Time	Area
19.246	145.616	Earthquake	131.6	6.0	1965-01-02 00:00:00	13:44:18	null
1.8630000000000002	127.352	Earthquake	80.0	5.8	1965-01-04 00:00:00	11:29:49	null
-20.579	-173.972	Earthquake	20.0	6.2	1965-01-05 00:00:00	18:05:58	null
-59.076	-23.557	Earthquake	15.0	5.8	1965-01-08 00:00:00	18:49:43	null
11.937999999999999	126.427	Earthquake	15.0	5.8	1965-01-09 00:00:00	13:32:50	null
-13.405	166.62900000000002	Earthquake	35.0	6.7	1965-01-10 00:00:00	13:36:32	null
27.357	87.867	Earthquake	20.0	5.9	1965-01-12 00:00:00	13:32:25	null
-13.309000000000001	166.21200000000002	Earthquake	35.0	6.0	1965-01-15 00:00:00	23:17:42	null
-56.452	-27.043000000000003	Earthquake	95.0	6.0	1965-01-16 00:00:00	11:32:37	null
-24.563000000000002	178.487	Earthquake	565.0	5.8	1965-01-17 00:00:00	10:43:17	null

only showing top 10 rows

显示数据部分行后发现Spark读取csv文件时将Date列读取成了1965-01-02 00:00:00的格式，因此还需要进一步对数据进行处理。需要对Date属性进行拆分，丢掉后面00:00:00的部分。最后将newDate命名为Date，这样数据就跟一开始预期的格式相一致了。

```python
from pyspark.sql.functions import split

df = df.withColumn("newDate",
                   split(df['Date'], " ")
[0]).drop("Date")
df = df.withColumnRenamed("newDate", "Date")
df.show(10)
```

Python

Latitude	Longitude	Type	Depth	Magnitude	Time	Area	Date
19.246	145.616	Earthquake	131.6	6.0	13:44:18	null	1965-01-02
1.8630000000000002	127.352	Earthquake	80.0	5.8	11:29:49	null	1965-01-04
-20.579	-173.972	Earthquake	20.0	6.2	18:05:58	null	1965-01-05
-59.076	-23.557	Earthquake	15.0	5.8	18:49:43	null	1965-01-08
11.937999999999999	126.427	Earthquake	15.0	5.8	13:32:50	null	1965-01-09
-13.405	166.62900000000002	Earthquake	35.0	6.7	13:36:32	null	1965-01-10
27.357	87.867	Earthquake	20.0	5.9	13:32:25	null	1965-01-12
-13.309000000000001	166.21200000000002	Earthquake	35.0	6.0	23:17:42	null	1965-01-15
-56.452	-27.043000000000003	Earthquake	95.0	6.0	11:32:37	null	1965-01-16
-24.563000000000002	178.487	Earthquake	565.0	5.8	10:43:17	null	1965-01-17

only showing top 10 rows

1. 将数据按年份、月份、日期计数

首先要对Date属性进行拆分，分别得到年月日的信息。

```python
attrsName = ['Year', 'Month', 'Day']
for i in range(len(attrsName)):
    df = df.withColumn(attrsName[i],
                       split(df['Date'], "-")
[i])

df.printSchema()
```

Python

```
root
 |— Latitude: double (nullable = true)
 |— Longitude: double (nullable = true)
 |— Type: string (nullable = true)
 |— Depth: double (nullable = true)
 |— Magnitude: double (nullable = true)
 |— Time: string (nullable = true)
 |— Area: string (nullable = true)
 |— Date: string (nullable = true)
 |— Year: string (nullable = true)
 |— Month: string (nullable = true)
 |— Day: string (nullable = true)
```

将Date属性拆分后发现年月日信息都是字符串类型，将其改为整型。

```python
for x in attrsName:
    df = df.withColumn(x, df[x].cast('int'))
```

接下来对数据分别按年月日计数并将结果保存到文件中，方便之后进行可视化。由于使用Spark自带的函数将DataFrame保存为csv文件时，文件会保存为文件夹，在本地读取时比较麻烦。因此使用toPandas()方法将Spark的DataFrame转换成pandas的DataFrame，再保存为csv文件，方便可视化时读取。

```python
countByYear = df.groupBy("Year").count().orderBy("Year")
countByYear.toPandas().to_csv("countByYear.csv",

encoding='utf-8',
                              index=False)

countByMonth = df.groupBy("Month").count().orderBy("Month")
countByMonth.toPandas().to_csv("countByMonth.csv",

encoding='utf-8',
```

```
                                        index=False)

countByDay =
df.groupBy("Day").count().orderBy("Day")
countByDay.toPandas().to_csv("countByDay.csv"
,

encoding='utf-8',
                                    index=False)
```
Python

2. 中国境内每个省份（海域）发生重大地震的次数

首先筛选出中国境内发生的重大地震信息，保存到文件中。

```
earthquakeC = df.filter("Area is not null")
earthquakeC.toPandas().to_csv("earthquakeC.csv
",

encoding='utf-8',
                                    index=False)
```
Python

按Area属性对数据进行分组计数，要注意只统计Area非空的数据。

```
countByArea =
earthquakeC.groupBy("Area").count()
countByArea.toPandas().to_csv("countByArea.csv
",

encoding='utf-8',
                                    index=False)
```
Python

3. 不同类型地震的数量

分别计算出中国境内和世界范围内的不同地震类型的数量。

```
countByTypeC = earthquakeC.groupBy("Type").count()
countByTypeC.toPandas().to_csv("countByTypeC.csv",

encoding='utf-8',
                                index=False)
```
Python

4. 震级前500的地震

根据震级进行降序排序，个人比较关注离现在更近的地震，因此再根据年份降序排列。由于使用take(500)之后返回一个RDD，其中每个元素都是Row对象。此时根据这个RDD生成DataFrame，再将其保存为csv文件。

```
mostPow = df.sort(df["Magnitude"].desc(), df["Year"].desc()).take(500)
mostPowDF = spark.createDataFrame(mostPow)
mostPowDF.toPandas().to_csv("mostPow.csv",
                            encoding='utf-8',
                            index=False)
```
Python

5. 震源深度前500的地震

当震源深度相同时，将震级更高的地震排在前面。

```
mostDeep = df.sort(df["Depth"].desc(), df["Magnitude"].desc()).take(500)
mostDeepDF = spark.createDataFrame(mostDeep)
mostDeepDF.toPandas().to_csv("mostDeep.csv",
                             encoding='utf-8',
                             index=False)
```
Python

6. 震级与震源深度的关系

将数据中的Depth和Magnitude拿出来，保存到文件中。

```
df.select(df["Magnitude"],
df["Depth"]).toPandas().to_csv("powDeep.csv",

encoding='utf-8',

index=False)
```
Python

至此数据分析基本完成，将df也保存到csv文件中，用于数据可视化。

```
df.toPandas().to_csv("earthquake1.csv",
                     encoding='utf-8',
                     index=False)
```
Python

四、 数据可视化

可视化工具使用plotly，可以绘制可交互的图表，支持保存为html格式的文件方便查看，并可以完美兼容Jupyter Notebook。另外，用WordCloud绘制词云。

1. 数据总览

首先将所有数据绘制到地图上。根据我们的常识，地震震级越高，地震的破坏力呈几何级数上升。为了画出更直观的图，使用`exp(Magnitude)/100`作为每个坐标的大小。画出的图都保存为html格式，方便查看。

```
import plotly.express as px
import plotly.io as pio

data = pd.read_csv('earthquake1.csv')

fig1 = px.scatter_geo(data,
                      color = data.Magnitude,
                      color_continuous_scale =
px.colors.sequential.Inferno,
```

```
                        lon = data.Longitude,
                        lat = data.Latitude,
                        hover_name = data.Type,
                        hover_data =
["Longitude",

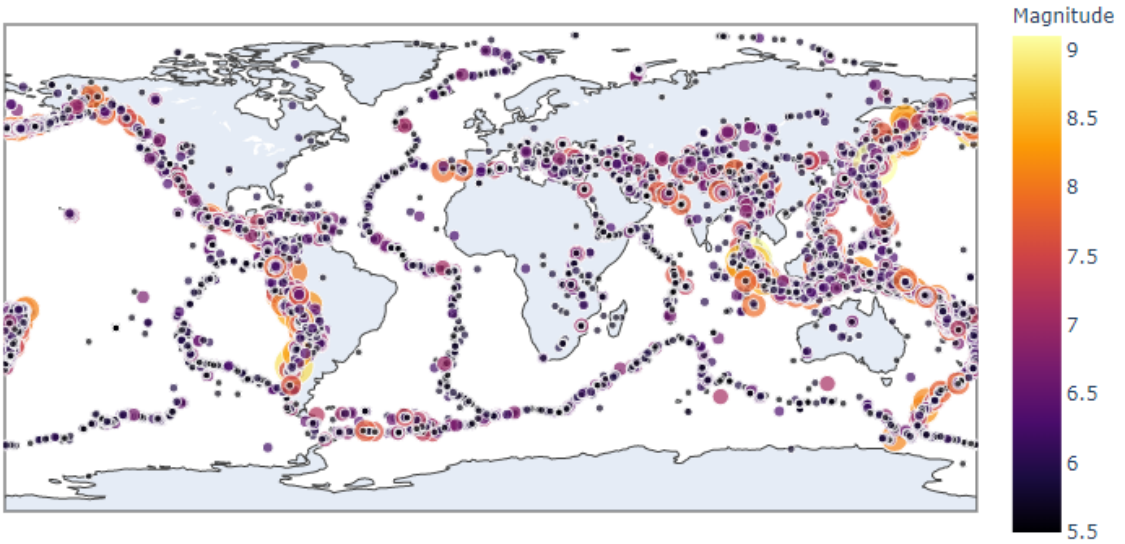
"Latitude",
                                                "Date",
                                                "Time",

"Magnitude",
                                                "Depth"
                                               ],
                        size =
np.exp(data.Magnitude)/100,
                        projection =
"equirectangular",
                        title = '1965-2016年全球
重大地震'
                                )
fig1.show()
pio.write_html(fig1, 'fig1.html')
```

Python

地图支持放大缩小，且鼠标指针停在某个点上时可以显示hover_data中设置的信息。可以清楚地看出重大地震主要分布在地震带上，但是同时在一张图上显示两万多个点导致这张图非常卡顿，浏览起来很不舒服。Plotly还支持根据某个数据为一帧生成动画，下面根据年份信息逐帧生成图片，不会卡顿且更加美观。

1965-2016年全球重大地震



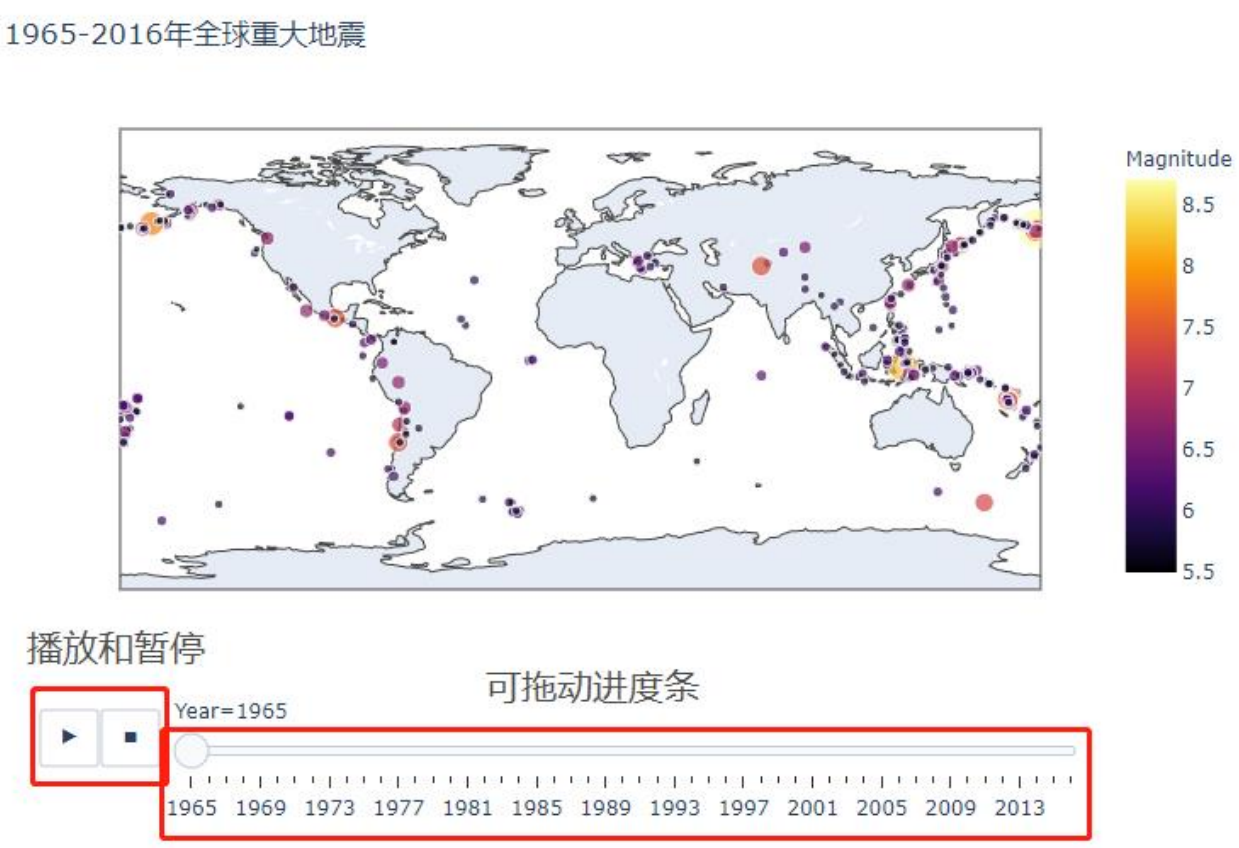

```
fig2 = px.scatter_geo(data,
                      color = data.Magnitude,
                      color_continuous_scale =
px.colors.sequential.Inferno,
                      lon = data.Longitude,
                      lat = data.Latitude,
                      animation_frame =
data.Year,
                      hover_name = data.Type,
                      hover_data =
["Longitude",

"Latitude",
                                              "Date",
                                              "Time",

"Magnitude",
                                              "Depth"
                                            ],
                       size =
np.exp(data.Magnitude)/100,
```

```
                          projection =
"equirectangular",
                          title = '1965-2016年全球
重大地震'
                          )
fig2.show()
pio.write_html(fig2, 'fig2.html')
```
Python

既可以根点击左下角的播放暂停按键也可以直接拖动下方的进度条控制进度。



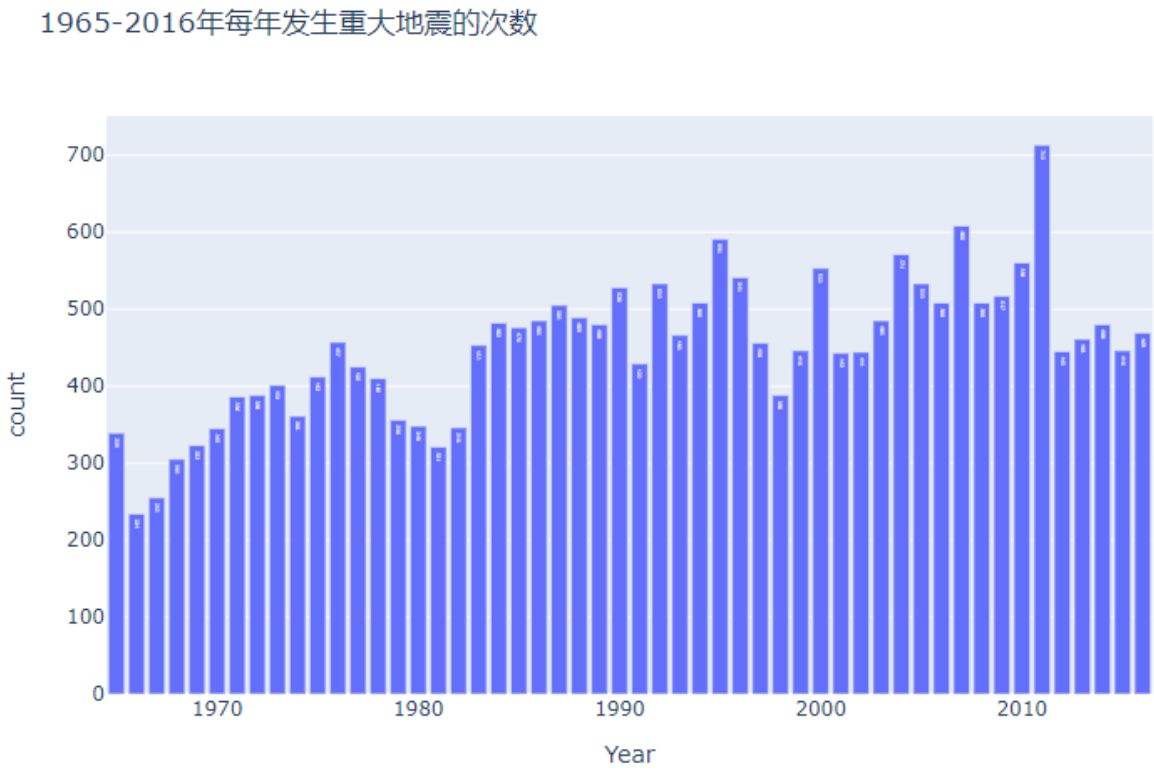
2. 每年份、月份、天份发生重大地震的次数

根据数据分析结果画出柱状图。

```
cntByYear = pd.read_csv('countByYear.csv')

fig3 = px.bar(cntByYear,
```

```
                x = "Year",
                y = "count",
                text = "count",
                title = '1965-2016年每年发生重大地震的次数'
                )
fig3.show()
pio.write_html(fig3, 'fig3.html')
```
Python

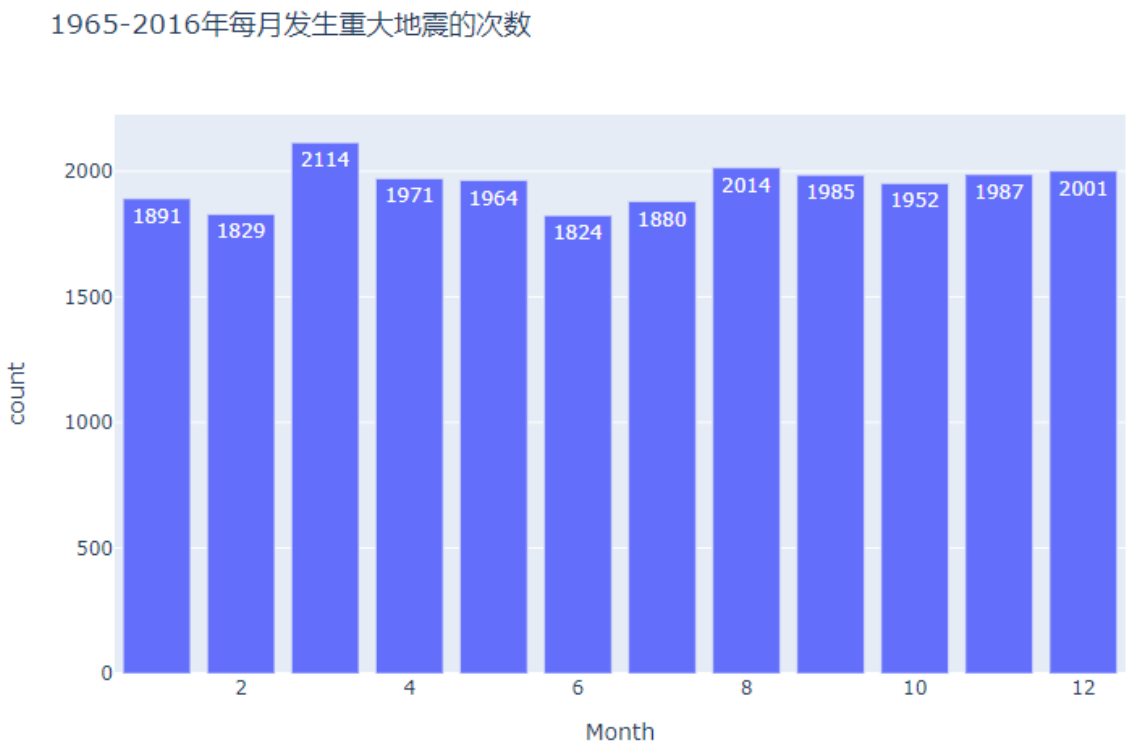


前十多年的地震次数明显少于之后的，可能是早期地震监测技术比较落后的原因。

```
cntByMonth = pd.read_csv('countByMonth.csv')

fig4 = px.bar(cntByMonth,
                x = "Month",
                y = "count",
                text = "count",
                title = '1965-2016年每月发生重大地震的次数'
```

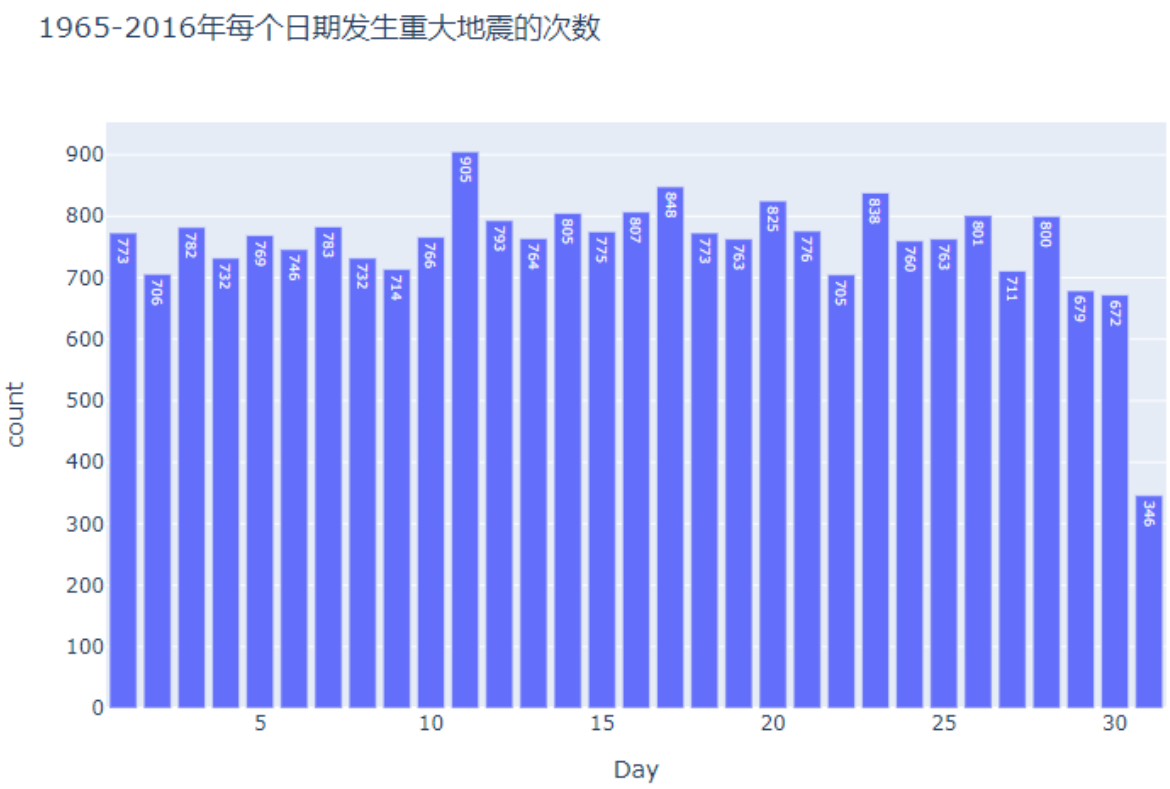
```
                )
fig4.show()
pio.write_html(fig4, 'fig4.html')
```
Python



每月发生的地震次数基本上一致。

```
cntByDay = pd.read_csv('countByDay.csv')

fig5 = px.bar(cntByDay,
              x = "Day",
              y = "count",
              text = "count",
              title = '1965-2016年每个日期发生重大地震的次数'
              )
fig5.show()
pio.write_html(fig5, 'fig5.html')
```
Python



由于有一半的月份没有31号，因此31号的次数明显较少。

3. 1955-2016年中国境内不同省份的重大地震次数

首先画出中国境内发生的重大地震。

```
dataC = pd.read_csv('earthquakeC.csv')

fig6 = px.scatter_geo(dataC,
                      color = dataC.Magnitude,
                      color_continuous_scale =
px.colors.sequential.Inferno,
                      lon = dataC.Longitude,
                      lat = dataC.Latitude,
                      scope = 'asia',
                      center = {'lon': 105.73,
'lat': 29.6},
```

```
                            hover_name =
dataC.Type,
                            hover_data =
["Longitude",

"Latitude",
                                                "Date",
                                                "Time",

"Magnitude",
                                                "Depth"
                                              ],
                            size =
np.exp(dataC.Magnitude)/100,
                            title = '1965-2016年中国
境内重大地震'
                                )
fig6.show()
pio.write_html(fig6, 'fig6.html')
```

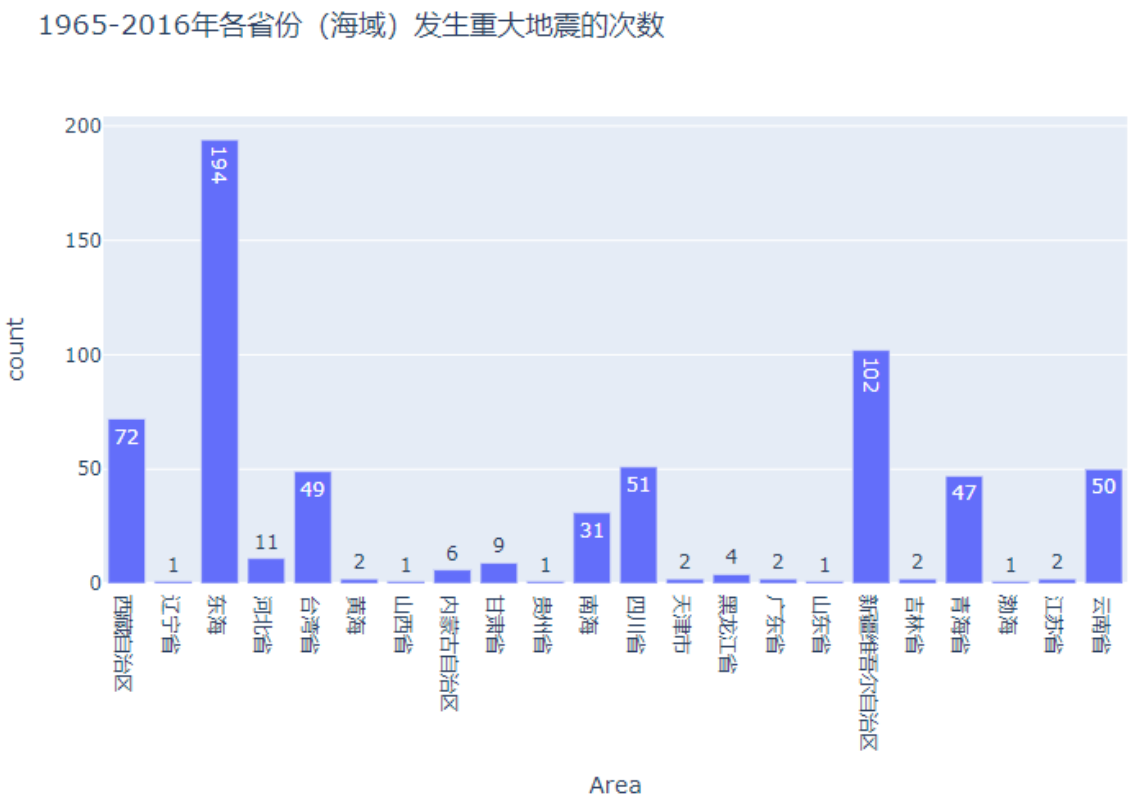
Python



再根据每个省份（海域）的数据生成柱状图和词云图。

```Python
cntByArea = pd.read_csv('countByArea.csv')

fig7 = px.bar(cntByArea,
              x = "Area",
              y = "count",
              text = "count",
              title = '1965-2016年各省份（海域）发生重大地震的次数'
              )
fig7.show()
pio.write_html(fig7, 'fig7.html')
```



生成词云图之前要下载一个中文字体，放入代码目录中，否则中文会出现乱码。可以去网上下载自己喜欢的中文字体，同时，本文也提供了微软雅黑的下载链接。百度网盘链接: https://pan.baidu.com/s/1xveNjm8ulH-rx_mx0lHnZg 提取码: figf

```
cntByArea = pd.read_csv('countByArea.csv')

fig7 = px.bar(cntByArea,
              x = "Area",
              y = "count",
              text = "count",
              title = '1965-2016年各省份（海域）发生重大地震的次数'
             )
fig7.show()
 pio.write_html(fig7, 'fig7.html')
```
Python



4. 震级前500的重大地震

```
pow500 = pd.read_csv('mostPow.csv')

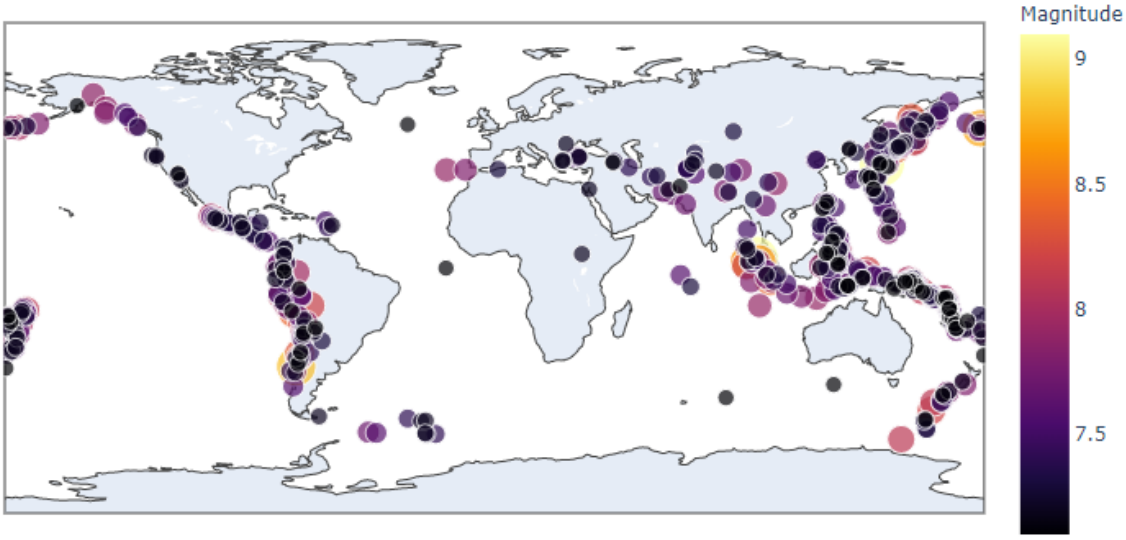
fig8 = px.scatter_geo(pow500,
```

```
                        color =
pow500.Magnitude,
                        color_continuous_scale =
px.colors.sequential.Inferno,
                        lon = pow500.Longitude,
                        lat = pow500.Latitude,
                        hover_name =
pow500.Type,
                        hover_data =
["Longitude",

"Latitude",
                                                "Date",
                                                "Time",

"Magnitude",
                                                "Depth"
                                               ],
                         size =
np.exp(pow500.Magnitude)/100,
                        title = '1965-2016年震级
前500的重大地震'
                                )
fig8.show()
pio.write_html(fig8, 'fig8.html')
```
Python

1965-2016年震级前500的重大地震



5. 震源深度前500的重大地震

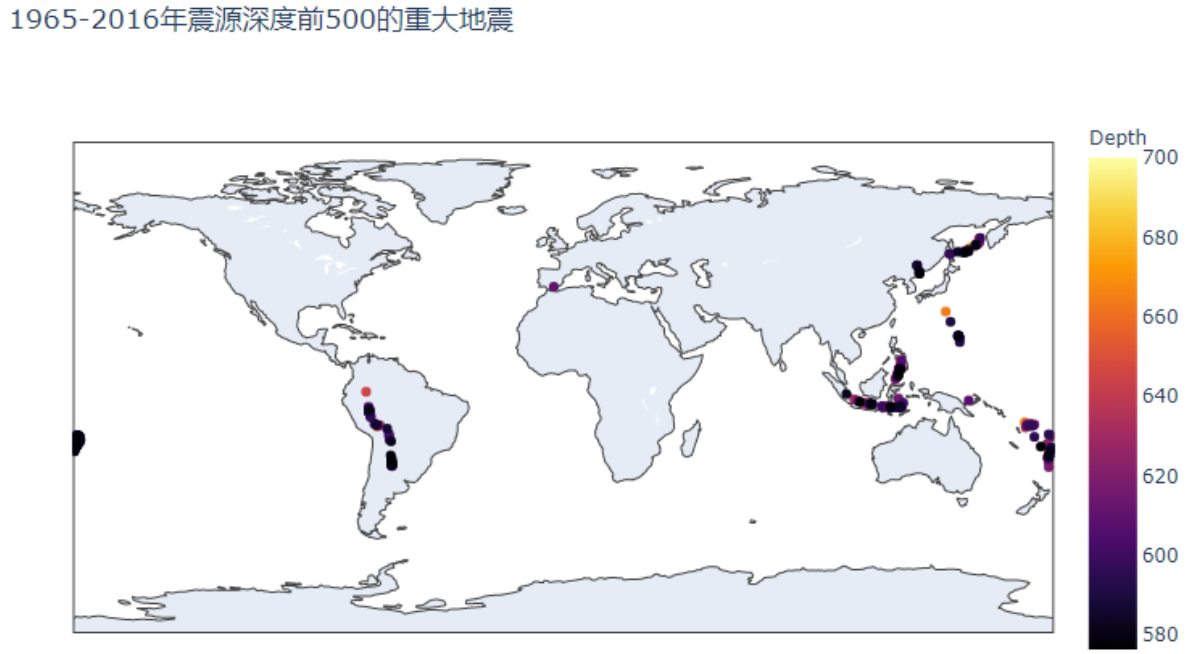
```
deep500 = pd.read_csv('mostDeep.csv')

fig9 = px.scatter_geo(deep500,
                      color = deep500.Depth,
                      color_continuous_scale =
px.colors.sequential.Inferno,
                      lon = deep500.Longitude,
                      lat = deep500.Latitude,
                      hover_name =
deep500.Type,
                      hover_data =
["Longitude",

"Latitude",
                                          "Date",
                                          "Time",

"Magnitude",
                                          "Depth"
                                        ],
```

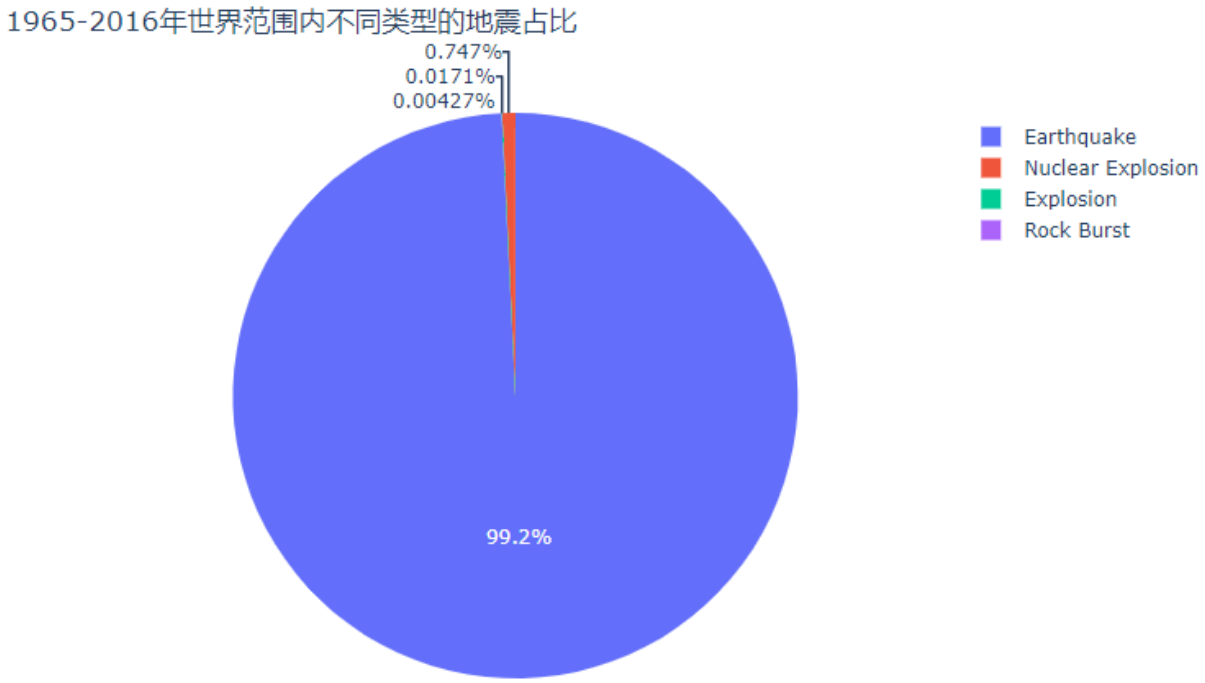
```
                              title = '1965-2016年震源深度前500的重大地震'
                              )
    fig9.show()
    pio.write_html(fig9, 'fig9.html')
```
Python



6. 不同类型的地震

根据数据画出饼状图。

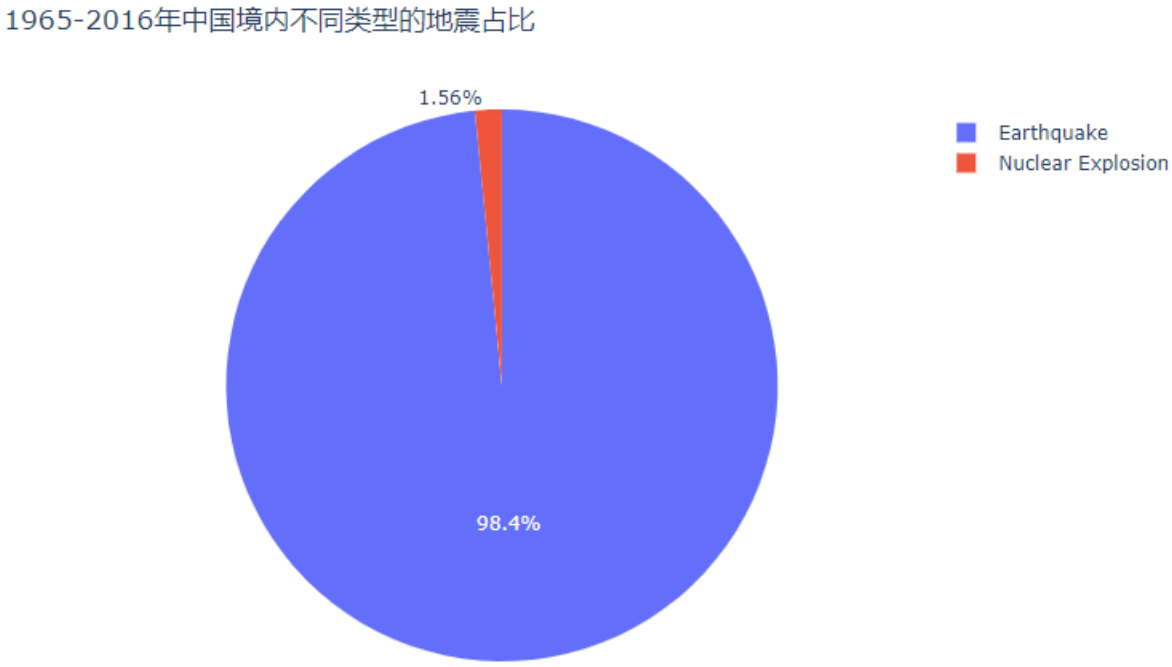
```
    fig10 = px.pie(cntByType,
                   names = "Type",
                   values = "count",
                   title = '1965-2016年世界范围内不同类型的地震占比'
                   )
    fig10.show()
    pio.write_html(fig10, 'fig10.html')
```
Python



可见99.2%是天然形成的地震，不到1%是核爆或者爆炸形成的，而岩爆现象则更加少见。同样地也可以画出中国境内不同类型的地震占比。

```Python
cntByTypeC = pd.read_csv('countByTypeC.csv')

fig11 = px.pie(cntByTypeC,
               names = "Type",
               values = "count",
               title = '1965-2016年中国境内不同类型的地震占比'
              )
fig11.show()
pio.write_html(fig11, 'fig11.html')
```

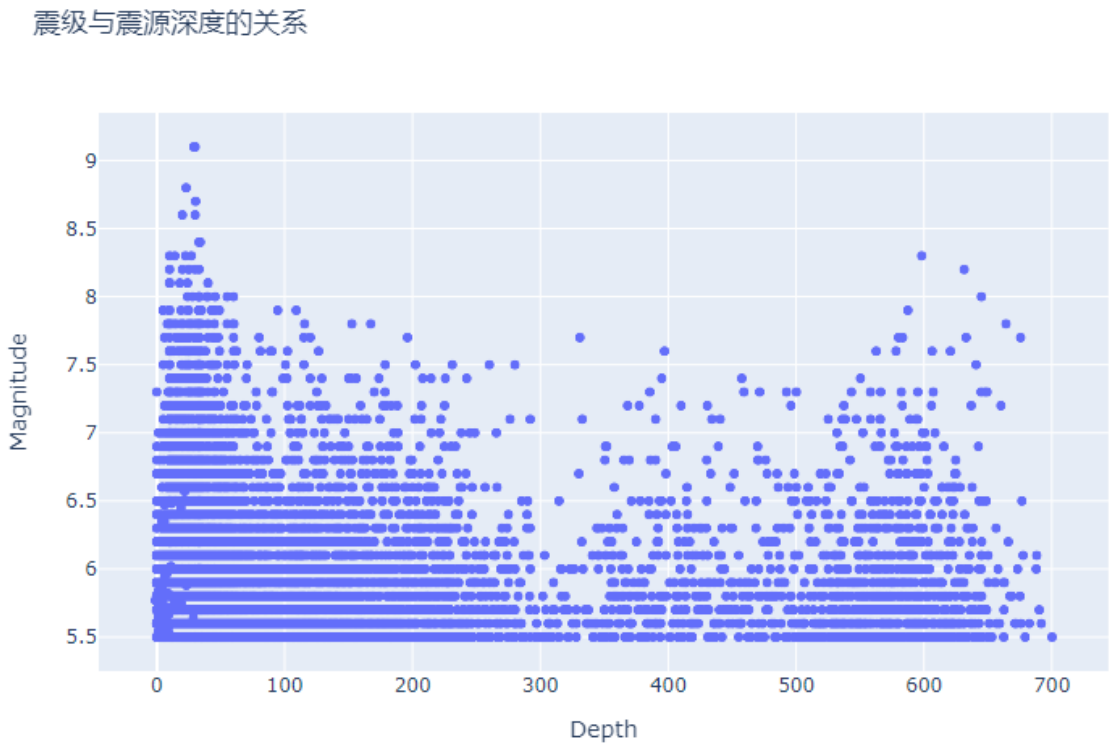


7. 震级和震源深度的关系

根据Depth和Magnitude画出散点图。

```Python
powAndDep = pd.read_csv('powDeep.csv')

fig12 = px.scatter(powAndDep,
                   x = "Depth",
                   y = "Magnitude",
                   title = '震级与震源深度的关系'
                  )
fig12.show()
pio.write_html(fig12, 'fig12.html')
```



可见震级8.0以上的地震大部分震源深度较浅，但从整体看地震的震级与震源深度的相关性不大。查资料发现震级只与该地震释放的能量有关，而通常震源深度越浅，造成的破坏就越大。